SALUS SECURITY

OCT 2026



CODE SECURITY ASSESSMENT

ZBT VESTING

Overview

Project Summary

- Name: ZBT Vesting
- Platform: EVM-compatible chains
- Language: Solidity
- Audit Range: See Appendix - 1

Project Dashboard

Application Summary

Name	ZBT Vesting
Version	v2
Type	Solidity
Dates	Oct 12 2026
Logs	Oct 10 2026, Oct 12 2026

Vulnerability Summary

Total High-Severity issues	0
Total Medium-Severity issues	0
Total Low-Severity issues	1
Total informational issues	1
Total	2

Contact

E-mail: support@salusec.io

SALUS

Risk Level Description

High Risk	The issue puts a large number of users' sensitive information at risk, or is reasonably likely to lead to catastrophic impact for clients' reputations or serious financial implications for clients and users.
Medium Risk	The issue puts a subset of users' sensitive information at risk, would be detrimental to the client's reputation if exploited, or is reasonably likely to lead to a moderate financial impact.
Low Risk	The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
Informational	The issue does not pose an immediate risk, but is relevant to security best practices or defense in depth.

SALUS

Content

Introduction **4**
1.1 About SALUS 4
1.2 Audit Breakdown 4
1.3 Disclaimer 4
Findings **5**
2.1 Summary of Findings 5
2.2 Notable Findings 6
1. Schedule Replacement Can Exceed the Intended Allocation 6
2.3 Informational Findings 7
2. Mapping Declaration Lacks Named Parameters for Improved Readability 7
Appendix **8**
Appendix 1 - Files in Scope 8

SALUS

Introduction

1.1 About SALUS

At Salus Security, we are in the business of trust.

We are dedicated to tackling the toughest security challenges facing the industry today. By building foundational trust in technology and infrastructure through security, we help clients to lead their respective industries and unlock their full Web3 potential.

Our team of security experts employ industry-leading proof-of-concept (PoC) methodology for demonstrating smart contract vulnerabilities, coupled with advanced red teaming capabilities and a stereoscopic vulnerability detection service, to deliver comprehensive security assessments that allow clients to stay ahead of the curve.

In addition to smart contract audits and red teaming, our Rapid Detection Service for smart contracts aims to make security accessible to all. This high calibre, yet cost-efficient, security tool has been designed to support a wide range of business needs including investment due diligence, security and code quality assessments, and code optimisation.

We are reachable on Telegram (https://t.me/salusec), Twitter (https://twitter.com/salus_sec), or Email (support@salusec.io).

1.2 Audit Breakdown

The objective was to evaluate the repository for security-related issues, code quality, and adherence to specifications and best practices. Possible issues we looked for included (but are not limited to):

- Risky external calls
- Integer overflow/underflow
- Transaction-ordering dependence
- Timestamp dependence
- Access control
- Call stack limits and mishandled exceptions
- Number rounding errors
- Centralization of power
- Logical oversights and denial of service
- Business logic specification
- Code clones, functionality duplication

1.3 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release and does not give any warranties on finding all possible security issues with the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues.

SALUS

Findings

2.1 Summary of Findings

ID	Title	Severity	Category	Status
1	Schedule Replacement Can Exceed the Intended Allocation	Low	Business Logic	Acknowledged
2	Mapping Declaration Lacks Named Parameters for Improved Readability	Informational	Code Quality	Resolved

SALUS

2.2 Notable Findings

Significant flaws that impact system confidentiality, integrity, or availability are listed below.

1. Schedule Replacement Can Exceed the Intended Allocation

Severity: Low	Category: Business Logic
Target: - src/ZBTVesting.sol	

Description

Replacing a schedule deletes its claim history and resets `claimed` to zero. A beneficiary can front-run removal to claim vested tokens, which are not deducted from the replacement allocation.

When increasing a cumulative allocation from 1,000 to 2,000 tokens, a beneficiary could first claim 500 and later receive another 2,000. This exceeds the intended total by 500 tokens.

src/ZBTVesting.sol:L226-L232

```
function _removeSchedule(address beneficiary) private {
    Schedule memory old = _schedules[beneficiary];
    if (old.total == 0) revert ScheduleMissing();
    // delete schedule.
    delete _schedules[beneficiary];
    emit ScheduleRemoved(beneficiary, old.total, old.claimed);
}
```

Recommendation

Add an owner-only function that increases `total` while preserving `claimed` and the original vesting timeline. Validate the increased total against the existing arithmetic bounds.

Status

This issue has been acknowledged by the team.

SALUS

2.3 Informational Findings

2. Mapping Declaration Lacks Named Parameters for Improved Readability

Severity: Informational	Category: Code Quality
Target: - src/ZBTVesting.sol	

Description

Since Solidity version 0.8.18, the language allows developers to add named key and value identifiers in mapping declarations using the syntax:

```
mapping(KeyType keyName => ValueType valueName)
```

This enhances code readability and clarifies how a mapping is intended to be used. In the `ZBTVesting` contract, the mapping `_schedules` is declared without named parameters, making the purpose of the key and value less obvious upon inspection.

src/ZBTVesting.sol:L82-L83

```
contract ZBTVesting is Ownable2Step, Pausable, ReentrancyGuard {
    /// @notice ZBT token distributed by this contract.
    IERC20 public immutable zbt;
    // @note Mapping Declaration Lacks Named Parameters for Improved Readability
    mapping(address => Schedule) private _schedules;
    ...
}
```

Recommendation

Consider updating the mapping declaration to explicitly name the key and value identifiers, enhancing readability and making the mapping's purpose clearer.

Status

This issue has been resolved by the team.

SALUS

Appendix

Appendix 1 - Files in Scope

This audit covered the following files:

File	SHA-1 hash
ZBTVesting.sol	80c8e20f2842b35fd6035402b8c70aa9099a8869

SALUS